# SIRIUS3

## Precision laser processing in one .NET platform

Design, edit, simulate and integrate machine control

**Product capabilities / v1.17**

# One platform from recipe to machine execution

A shared document and device model connects the full workflow

**Prepare**

CAD, text and codes

**Edit**

Geometry and process conditions

**Inspect**

3D view and path simulation

**Execute**

RTC, laser and motion integration

## Reusable integration

Scanner, laser, power meter and remote interfaces

## Consistent operation

One recipe model across editing and execution

## Room to customize

Your equipment logic and your application UI

# More ways to edit, import and connect

Recent releases expand the work you can keep inside Sirius3

| Release | Added capability | Customer value |
| --- | --- | --- |
| **1.17** | Grips, FontEditor, PDF images | Edit geometry and fonts close to the recipe |
| **1.16.1** | IClippable, SVG, 3MF, PCHIP | Divide areas and bring in more source data |
| **1.15** | HiDPI, textured mesh previews | Inspect detail across modern displays |
| **1.14.5+** | Native WPF and embedded MCP | Connect application UI and AI workflows |

**Current Release support includes both WPF and MCP**

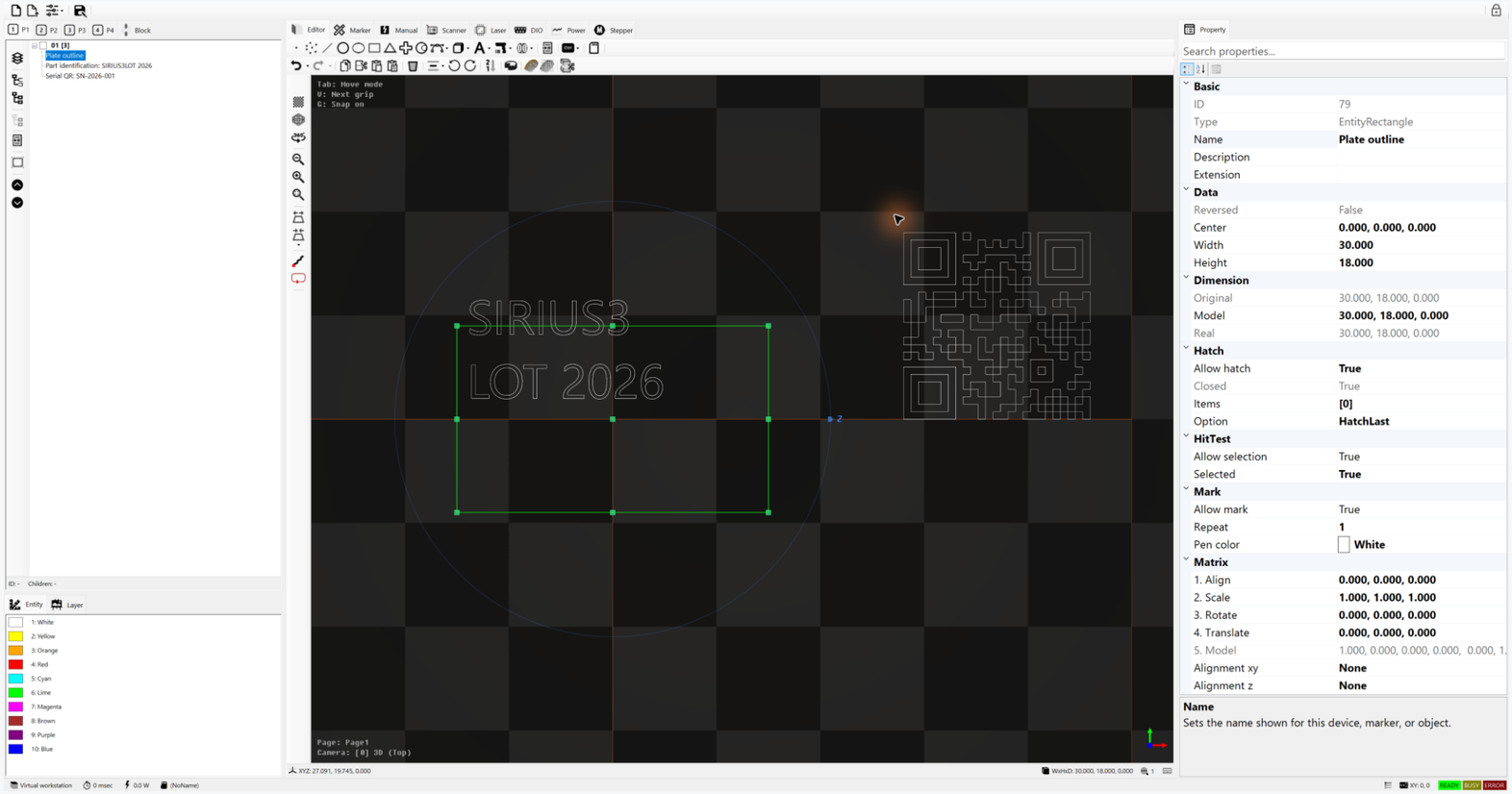
# A fit for marking, microprocessing and machine OEMs

Match the software workflow to the process challenge

| Application need | Sirius3 workflow | Integration focus |
| --- | --- | --- |
| **Part identification** | Text, 1D/2D codes, dynamic data | Code size, data source and readability |
| **Pattern and area processing** | Vector import, multiple hatches, clipping | Topology, spacing and field layout |
| **3D preparation** | Mesh preview, slicing and surface clouds | Units, Z range and correction method |
| **Moving or large workpieces** | MoF or coordinated syncAXIS | Encoder, motion and vendor prerequisites |
| **Equipment integration** | WinForms/WPF, Remote and MCP | Device registration and host workflow |

# The recipe stays visible while you edit

A document tree, drawing view and PropertyGrid work together




Running editor with a selected object and populated properties

## Four pages

Layers, groups, reusable blocks and pens

## Direct property editing

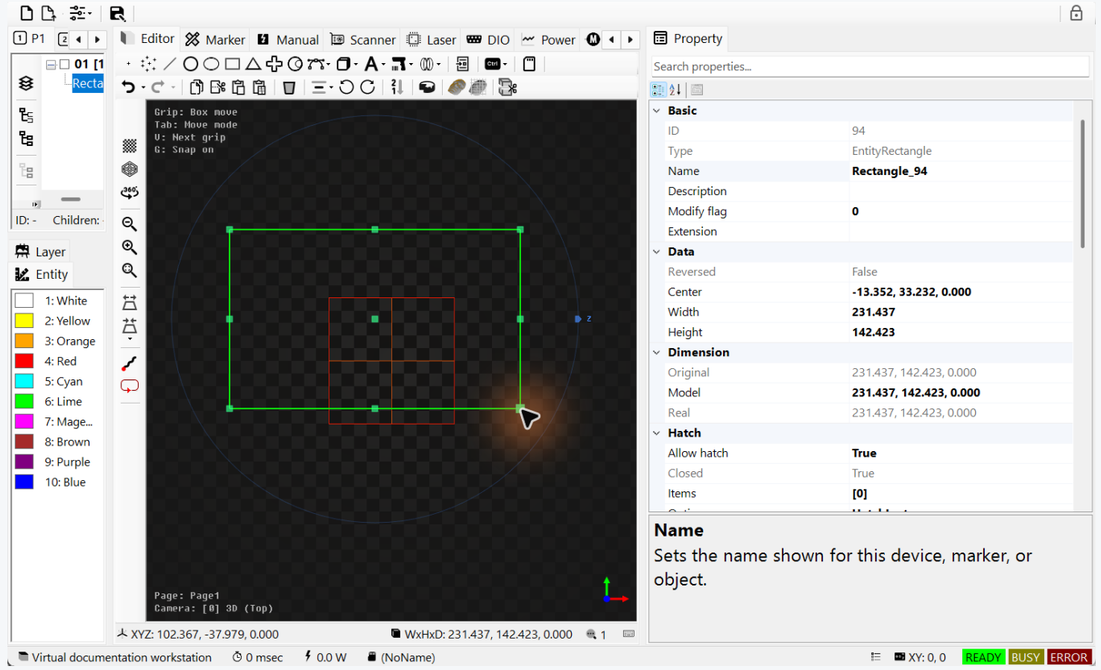
Search, inspect and adjust the selected object

## Undo/Redo

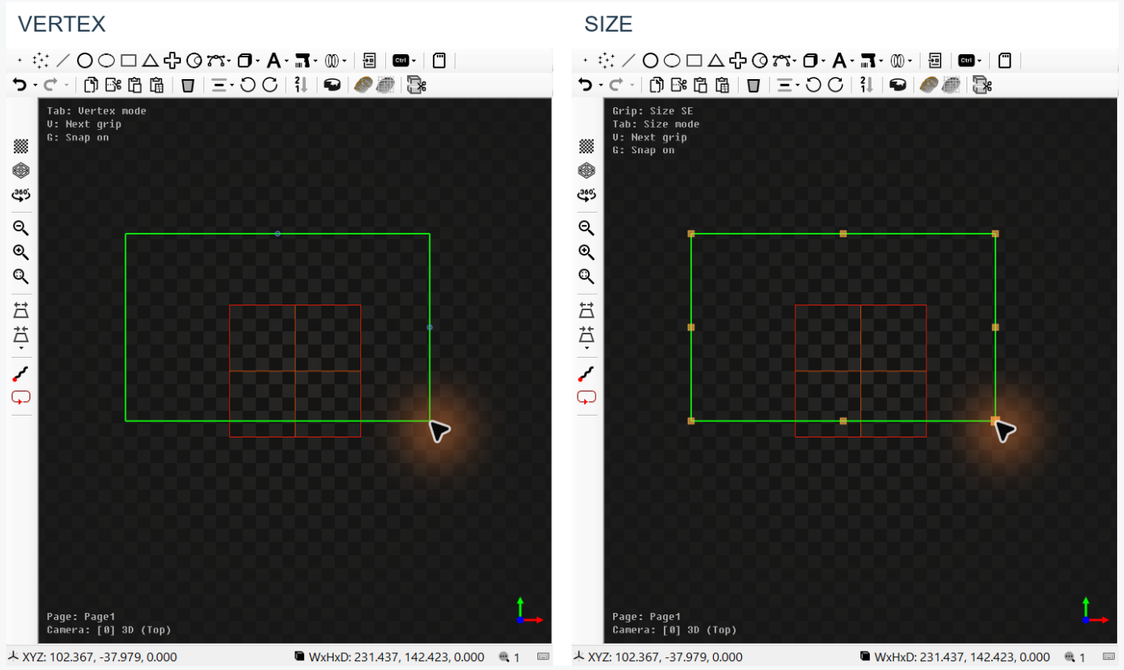
Review and revise document changes

# Shape changes happen directly on the canvas

Interactive creation and IGrippable editing keep the work visual




Create a shape, then inspect its dimensions

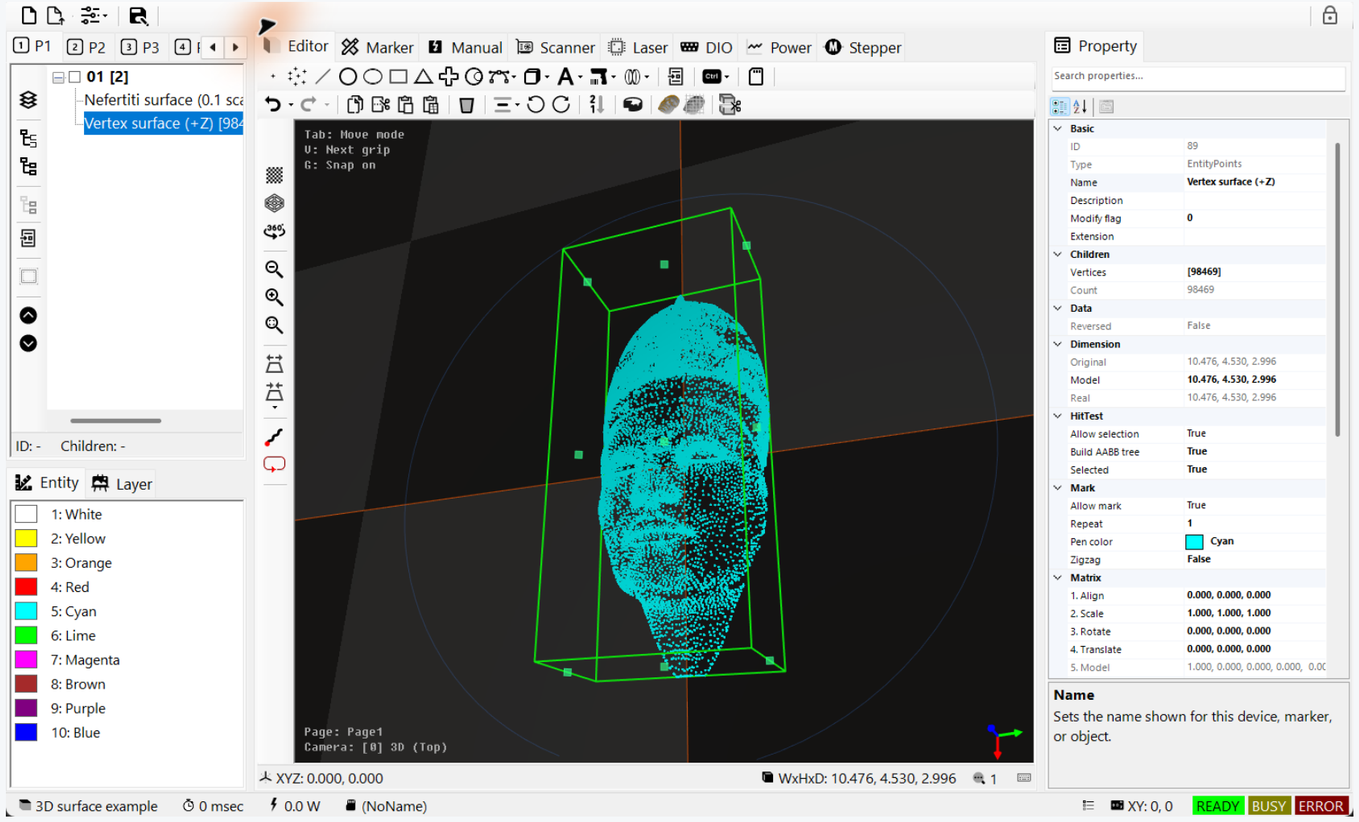



Vertex and size handles for direct editing

**Move, edit vertices, resize and rotate with Undo/Redo**

# Dense geometry remains visible and editable

Render dense 2D/3D data and edit the original vertices




Documented surface-cloud example with 98,469 points

## GPU rendering

OpenGL 3.3+ displays 2D/3D geometry

## Spatial picking

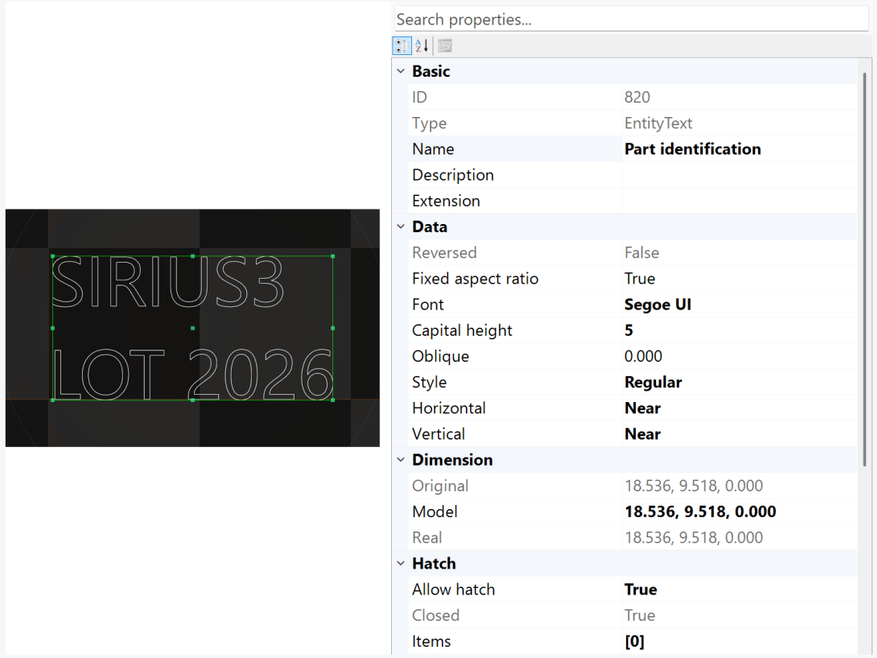
AABB acceleration for geometry selection

## Original vertices

Edit supported vertices with grips and Undo/Redo

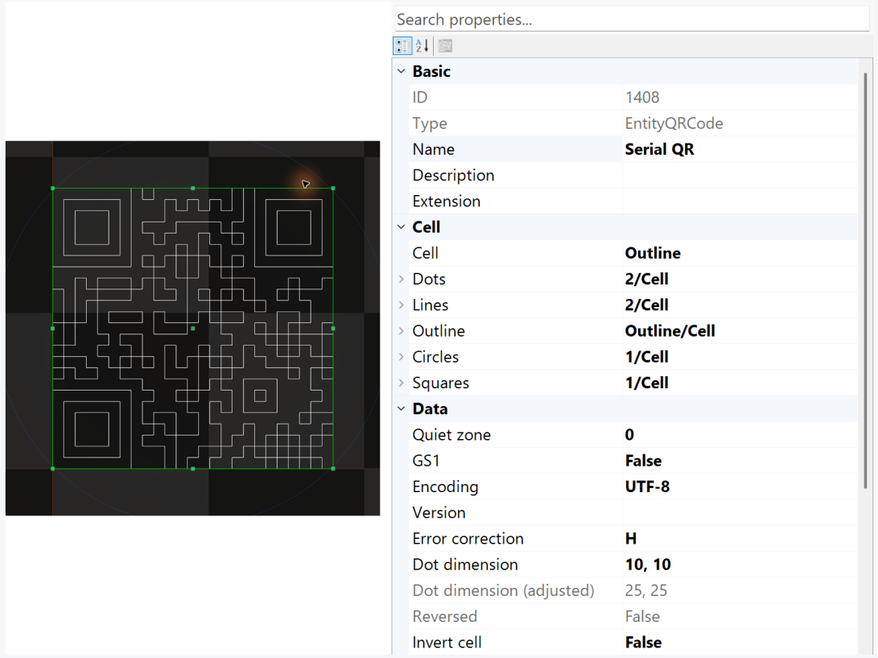
# Production data becomes text and codes

Serial numbers, lots and linked data belong in the recipe




Vector text with conversion settings

**1D, QR, DataMatrix, PDF417 and Aztec**

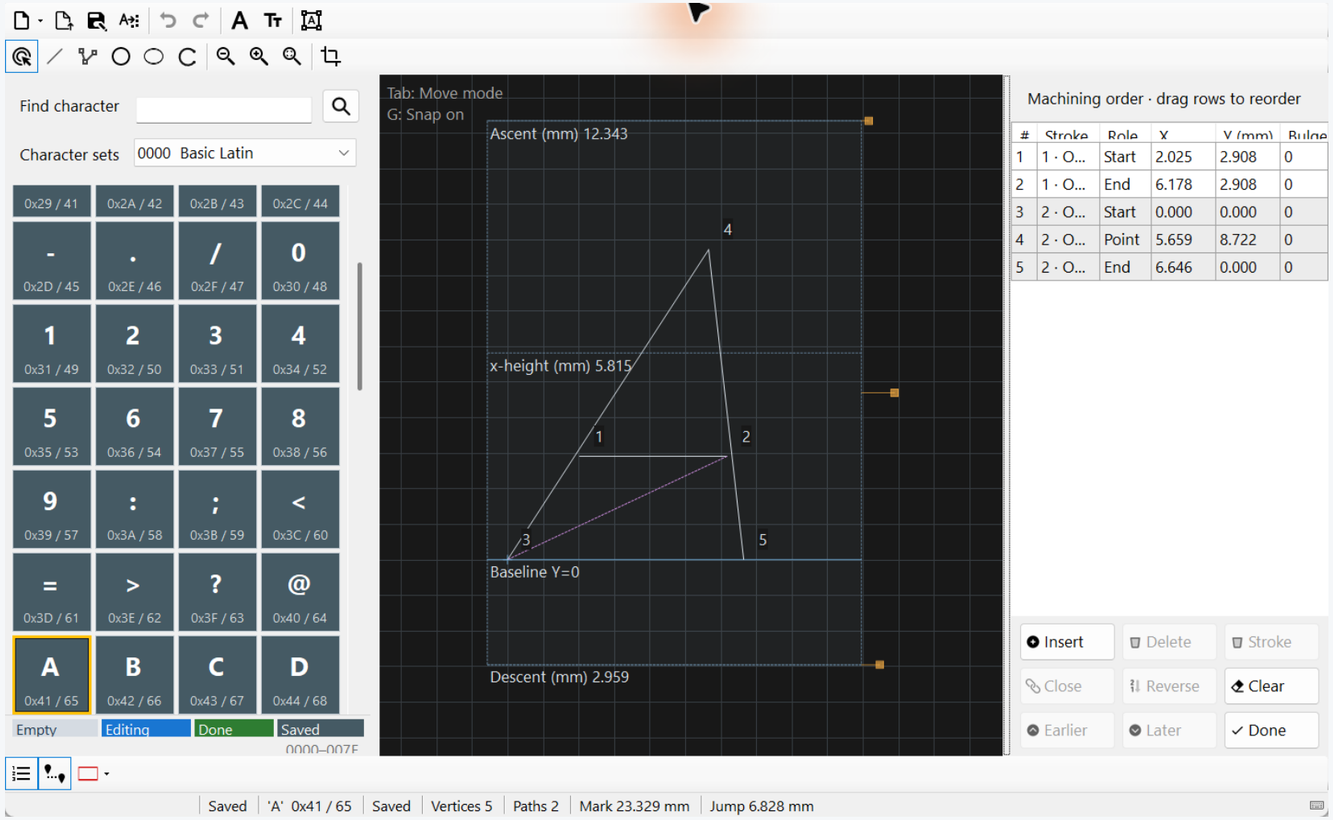



2D code with outline and hatch settings

**Event, file, offset, linked entity or C# Script**

# Fonts can follow the process requirement

FontEditor edits glyph paths and metrics independently of machine control




Actual glyph editing with baseline and font metrics

## Edit and preview

Glyphs, paths, order and text preview

## Production font formats

CXF, LFF, FNT and DOT support

## Installed-font conversion

Convert TrueType outlines or DOT pixels

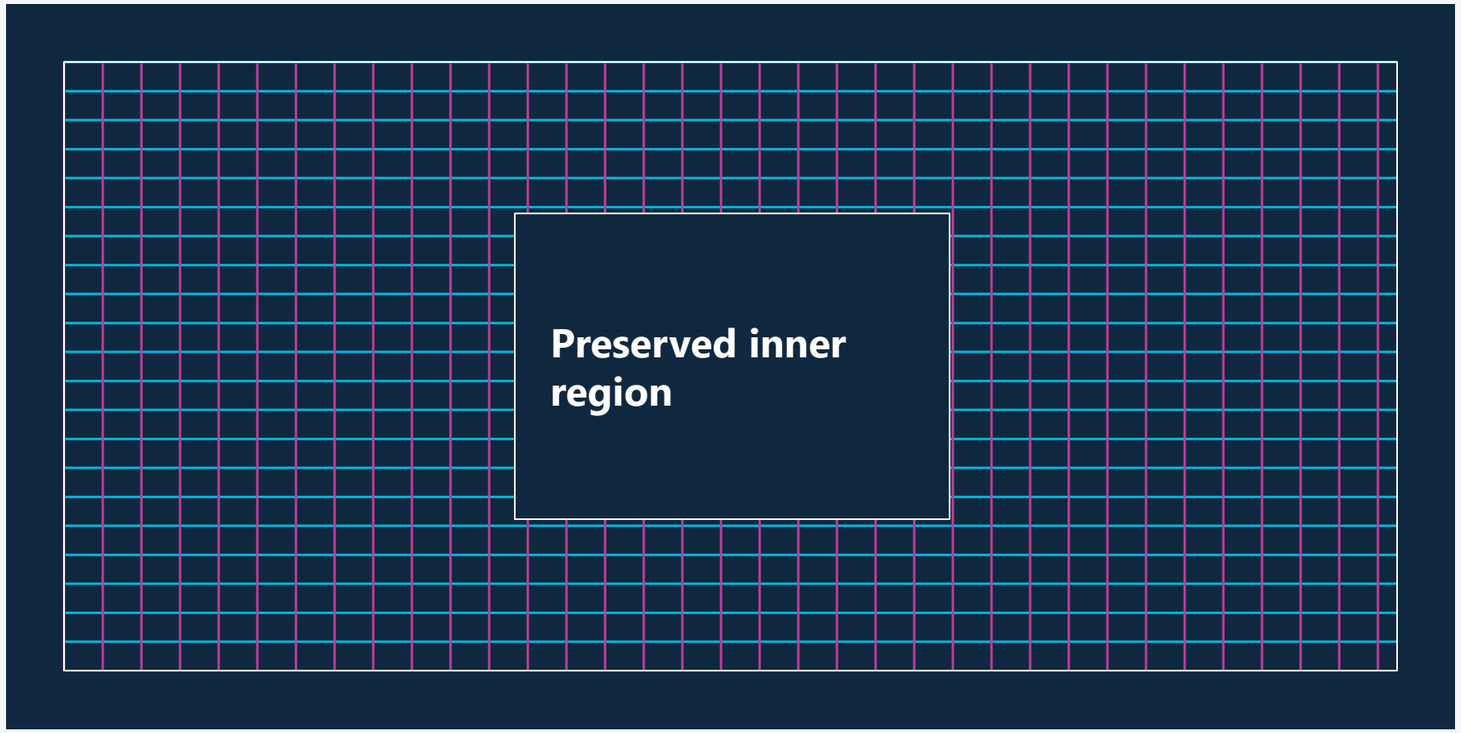
# More source formats reach the same recipe

Bring in CAD, graphics, machine paths and 3D models

| Source | Supported examples | Result and scope |
| --- | --- | --- |
| **2D CAD / vector** | DXF, DWG, SVG, HPGL/PLT | Editable paths and shapes |
| **PCB / drill data** | Gerber, Excellon | Geometry from detected file content |
| **Machine paths** | G-code / NGC | Paths with source metadata |
| **3D models** | STL, OBJ, PLY, STEP, 3MF | Meshes with available colors/textures |
| **Documents / images** | PDF, BMP, PNG, JPG, ZPL | Raster content for image workflows |

# Fill patterns stay connected to geometry

Multiple hatches combine with explicit contour and hole handling




Multiple hatch patterns, schematic

## Pattern control

Line and polygon patterns, angle, pitch and order

## Topology-aware fill

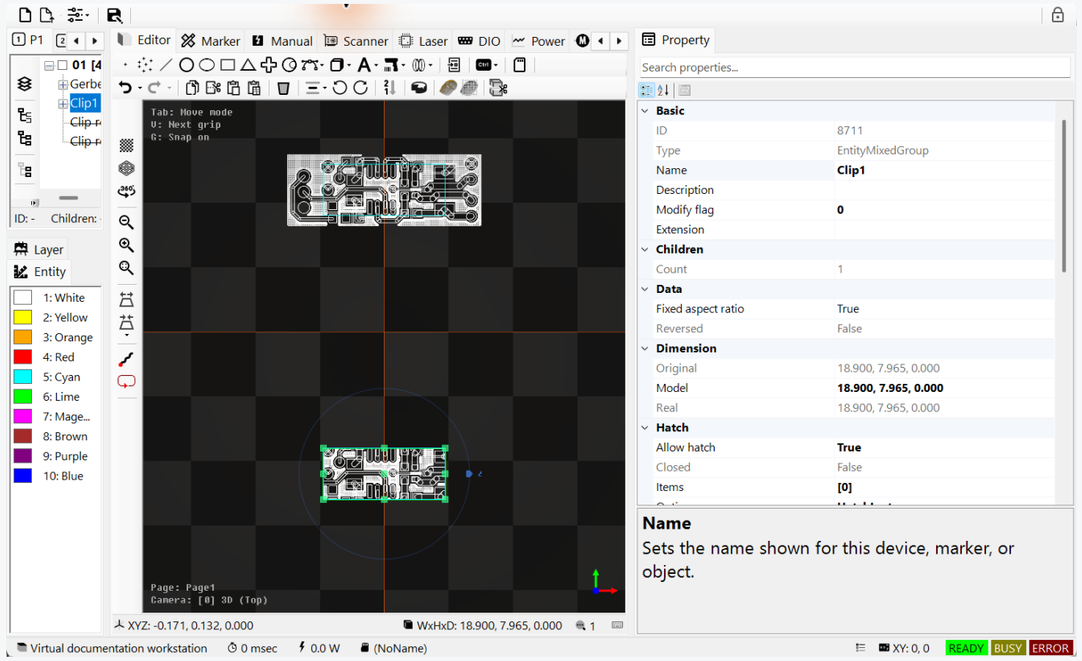
Closed regions, nested holes and connected cells

## Process sequence

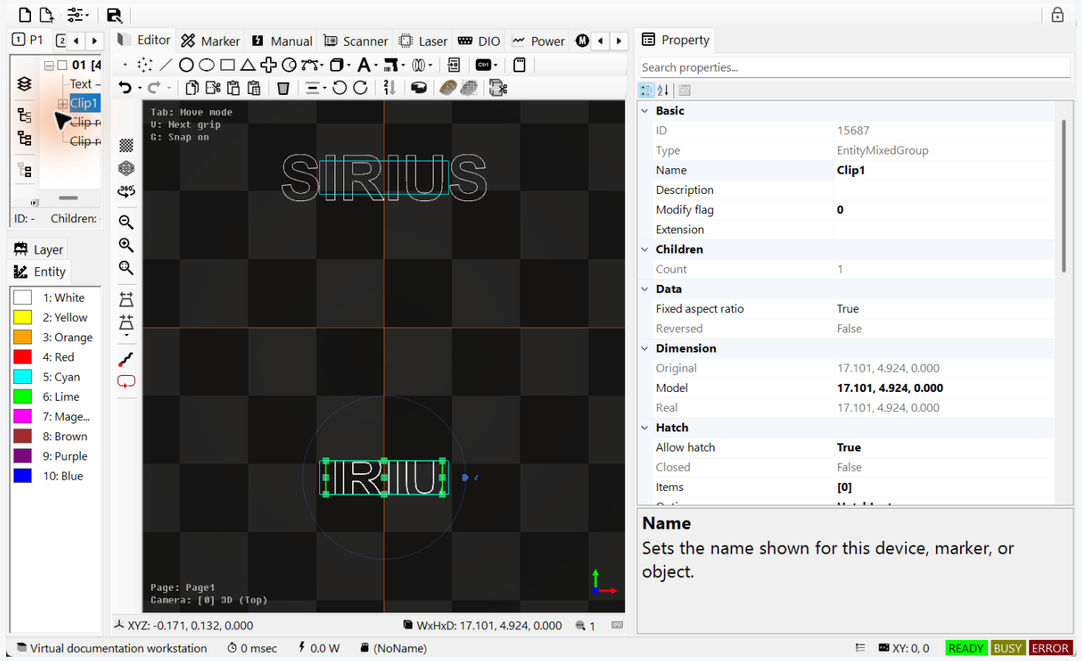
Hatch first, hatch last or hatch only

# Large patterns become manageable processing regions

IClippable and ActClip crop real paths with document Undo/Redo




Gerber source above, clipped result below

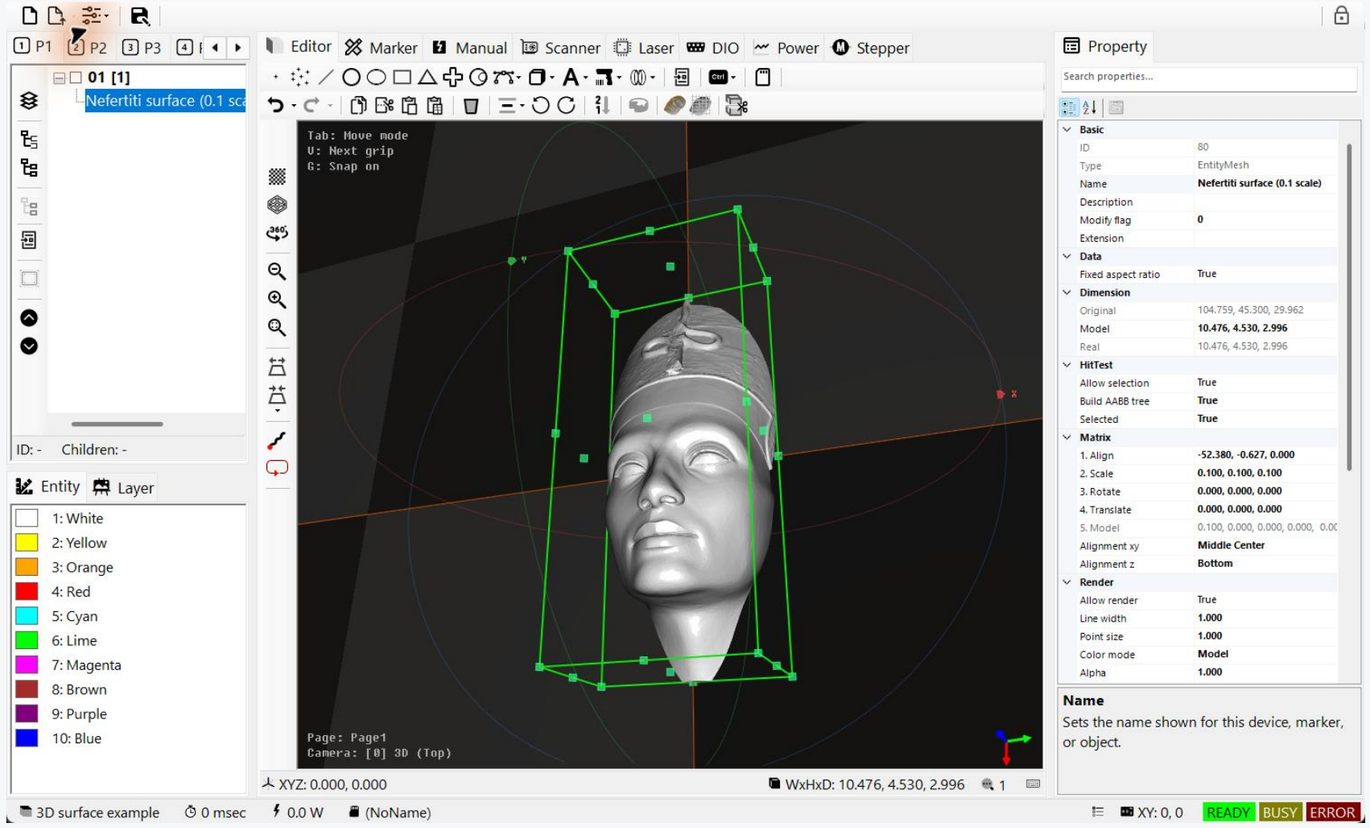



Text source and clipped machining paths

**Paths or closed regions, with the option to keep the source**

# 3D models enter the editing workflow

Inspect placement, dimensions and surface detail before path preparation




Selected mesh with dimensions and transformations

## Six built-in cameras

One orthographic and five perspective views

## Visual context

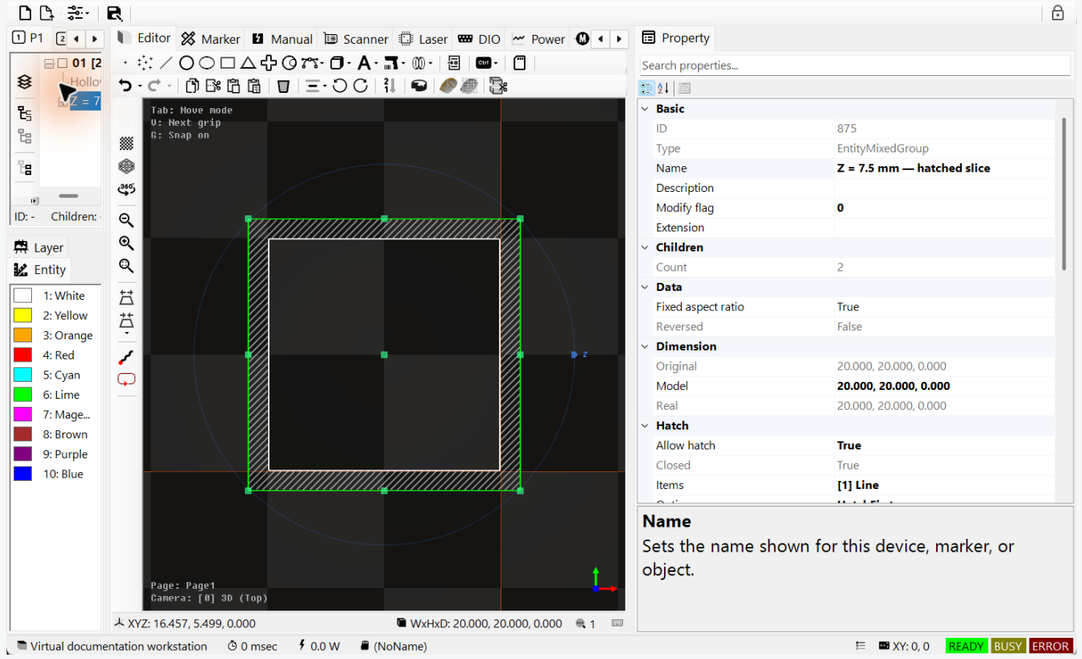
Available mesh colors and diffuse textures

## Editable geometry

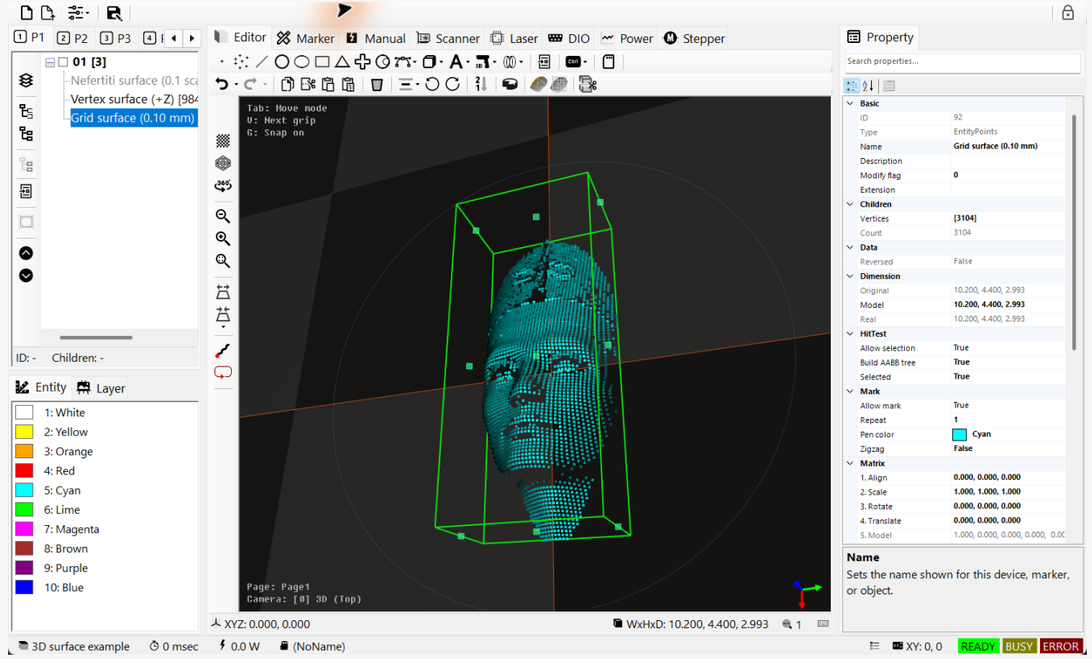
Transforms and applicable vertex/control grips

# From a 3D surface to processing data

Slicing and surface-cloud extraction serve different preparation tasks




Cross-section contours and hatch at a selected Z



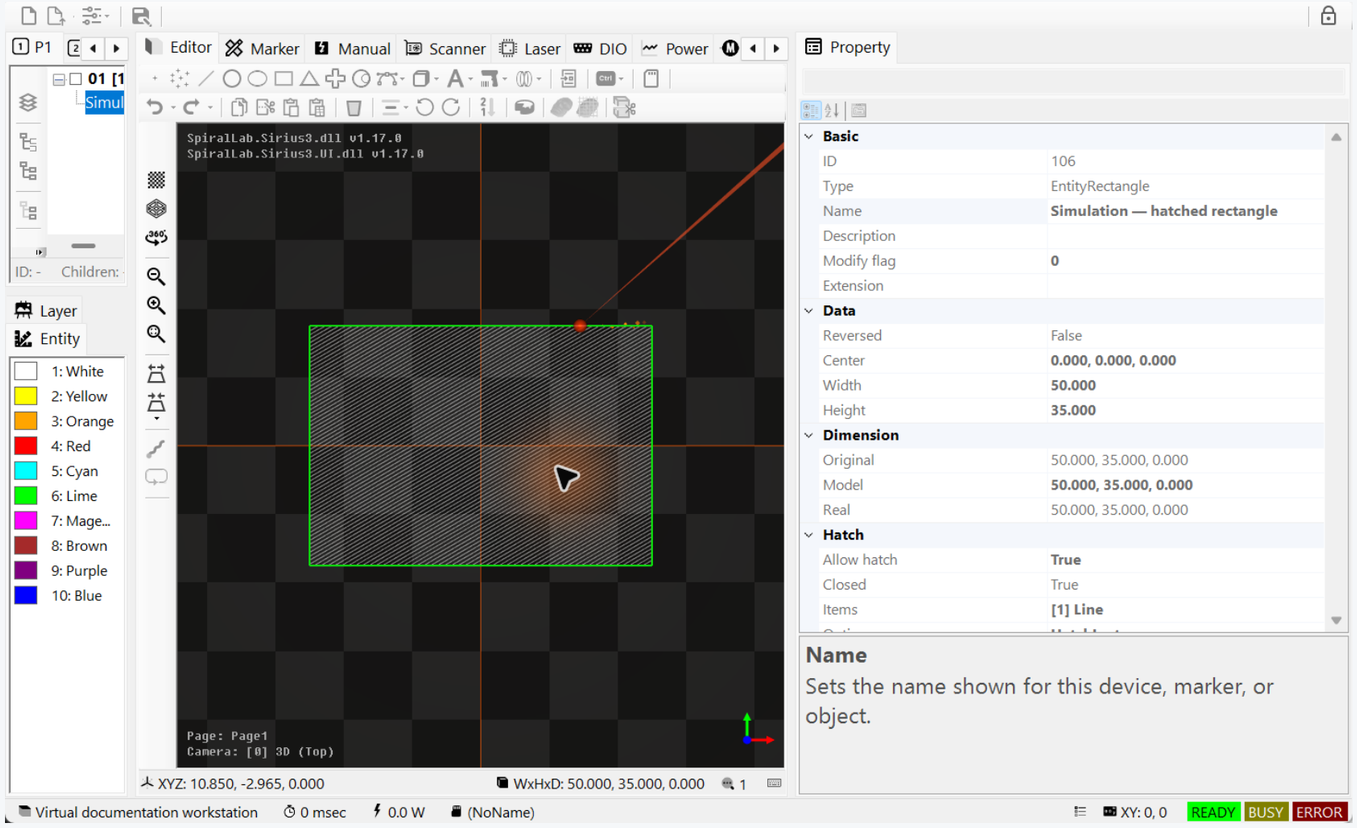

Grid-cloud extraction for freeform correction input

**Slice, inspect contours, then apply hatching**

**Use extracted XYZ data in the calibration workflow**

# The path can be reviewed before execution

Selected geometry becomes a visual mark-and-jump simulation




Actual path-simulation view with selected geometry

## Visible sequence

Review outlines, hatch traversal and travel moves

## Adjustable playback

Fast, normal and slow simulation

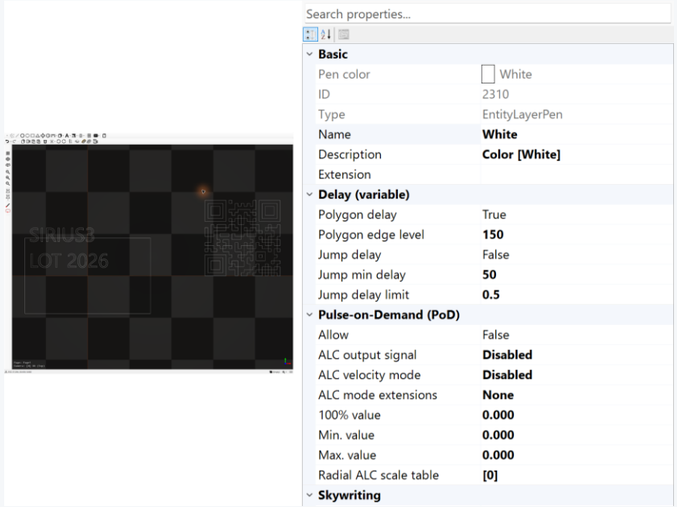
## Clear visual feedback

Screen-sized markers and beam effects

# Process conditions follow the recipe structure

Entity pens and layer pens separate local conditions from layer behavior

| Control level | Typical settings | Purpose |
| --- | --- | --- |
| **Entity pen** | Power, frequency, speed and timing | Apply process conditions to geometry |
| **Layer pen** | Sky Writing, ALC and variable delays | Control layer-wide scanner behavior |
| **Marker / offsets** | Layer order, placements and repetition | Build the complete job sequence |




Selected layer settings

## Geometry, process conditions and placement stay connected

# A controller family can share one software foundation

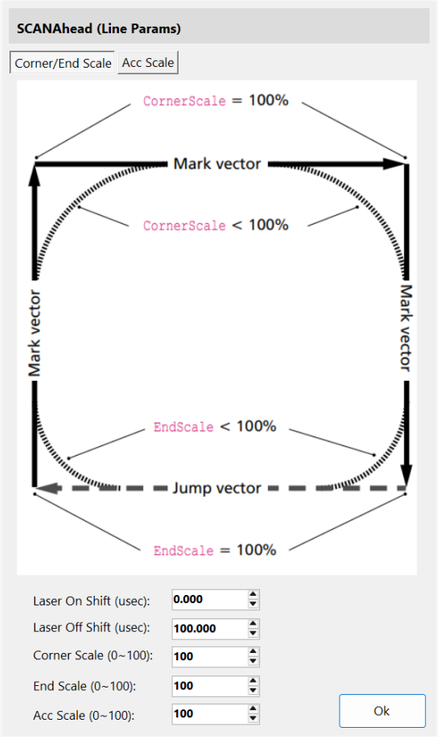
RTC4/4e, RTC5 and RTC6/6e integrations with generation-specific features

| Capability | RTC4 / RTC4e | RTC5 | RTC6 / RTC6e |
|---|---|---|---|
| **Sirius3 integration** | Supported | Supported | Supported |
| **Sky Writing** | Not supported | Modes 1-3 | Modes 1-4 |
| **SCANahead / Auto Delays** | Not supported | Not supported | Matching scan head and options |
| **Coordinated syncAXIS** | Not applicable | Not applicable | RTC6-based system option |

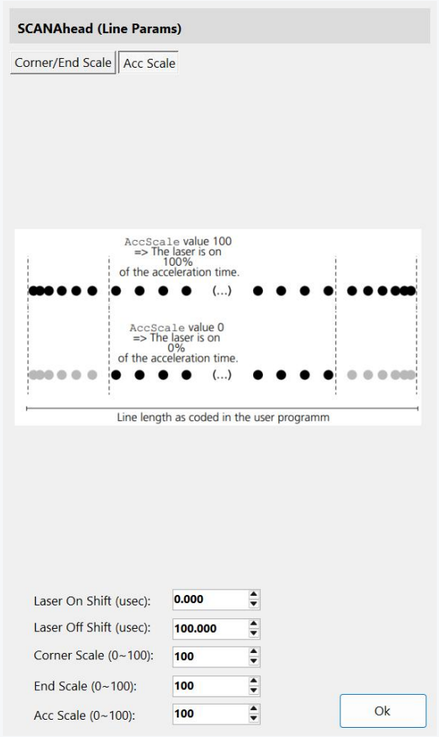
Feature availability also depends on firmware, scan head, vendor runtime and licensed options.

# SCANahead support extends path and timing control

For a compatible RTC6 and scan-head configuration




Corner and End scale settings



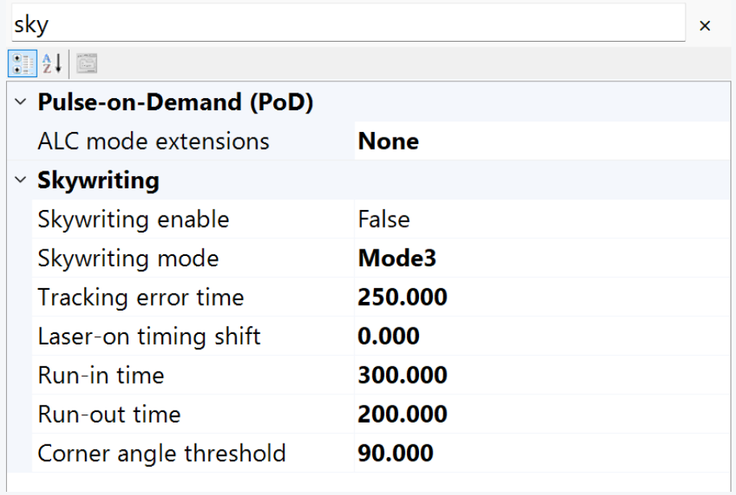

AccScale settings

| Capability | What it controls |
|---|---|
| **Corner / End** | Scanner path behavior at corners and ends |
| **AccScale** | Emission window during acceleration |
| **Auto Delays** | Dynamic, speed-dependent timing |
| **SDC / ALC** | Supported spot-spacing and output combinations |

**Compatible RTC6, scan head, firmware and options required**

# Timing tools address the shape of the path

Sky Writing and variable delays target different motion conditions




Sky Writing settings in the editor

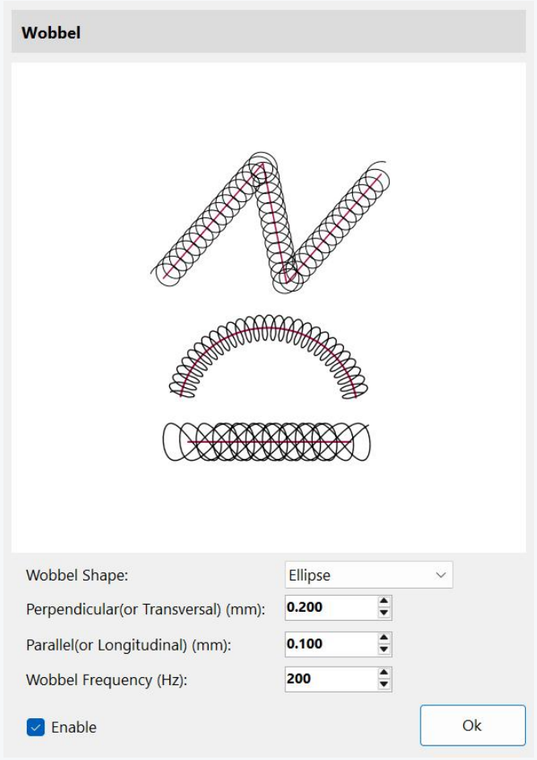
| Tool | Where it applies | Purpose |
|---|---|---|
| **Sky Writing** | Approach and exit around mark vectors | Allow motion outside the marking segment |
| **Variable jump delay** | Different jump distances | Adapt settling time to the jump |
| **Variable polygon delay** | Corners between mark segments | Adapt timing to the deflection angle |

## Settings follow the controller, head and actual process

# Energy control can follow motion and position

ALC, defined-vector ramps and Wobbel have distinct roles

| Method | Input relationship | Process role |
|---|---|---|
| **Speed-dependent ALC** | Output follows configured velocity relation | Adapt the output command to motion |
| **Position-dependent ALC** | Table by radial distance and factor | Adjust the command across the field |
| **Defined-vector ramp** | Output along vector distance | Shape output along a defined path |
| **Wobbel** | Oscillation over the nominal path | Create a superimposed scan pattern |



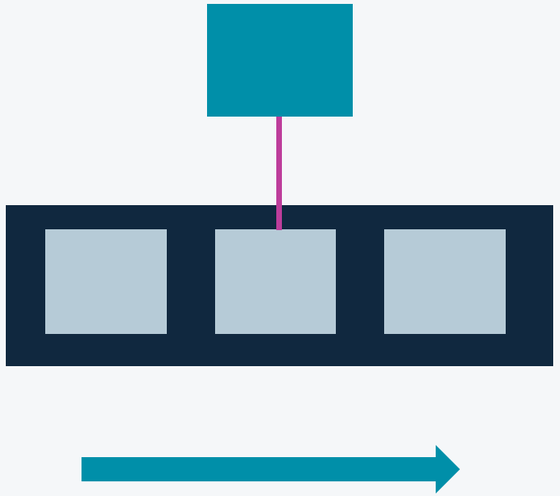

Wobbel parameter editor

Choose settings within the installed head and laser limits.

# Moving workpieces can share the marking workflow

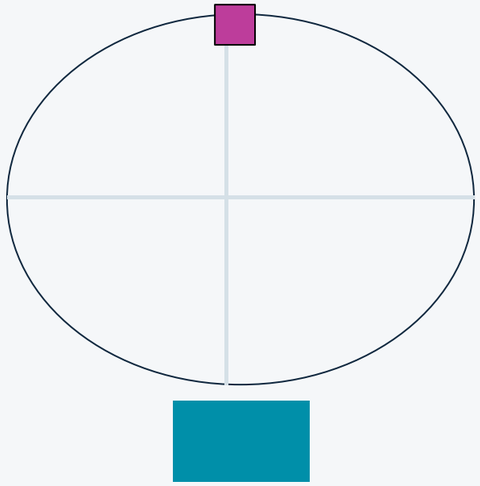
MoF uses encoder information for moving-coordinate compensation

## Linear tracking



Encoder displacement

## Rotary tracking



Encoder angle

Conceptual motion diagrams

### Linear and rotary

Configure the motion model for the installed encoder

### Recipe synchronization

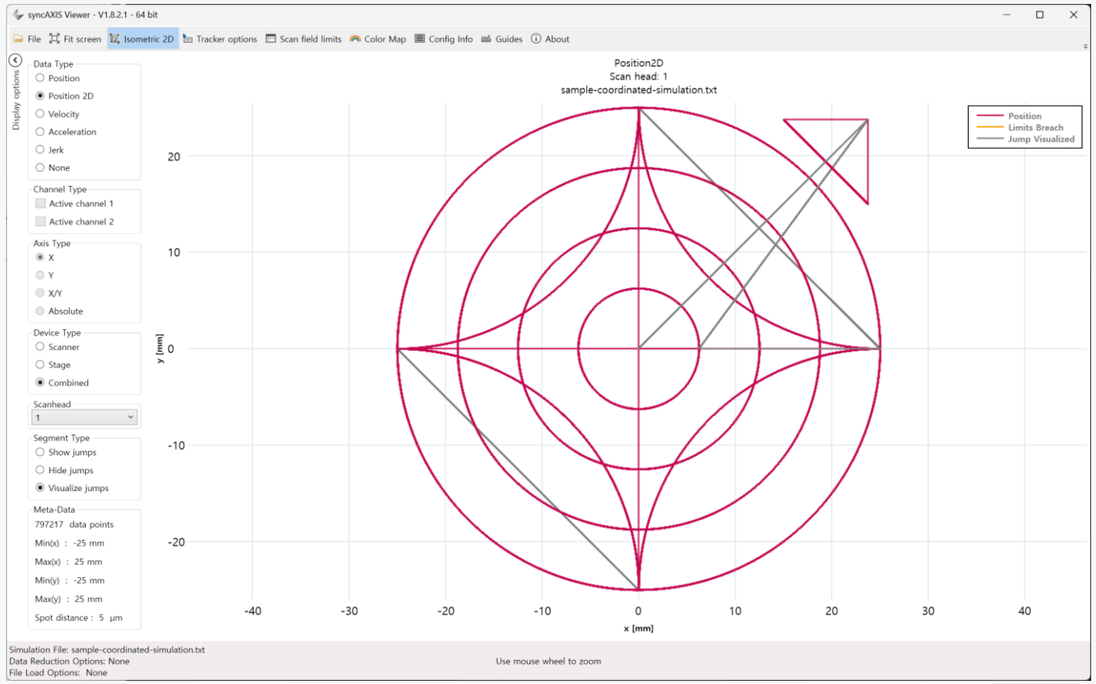
Combine tracking, waits and placement logic

### An extension option

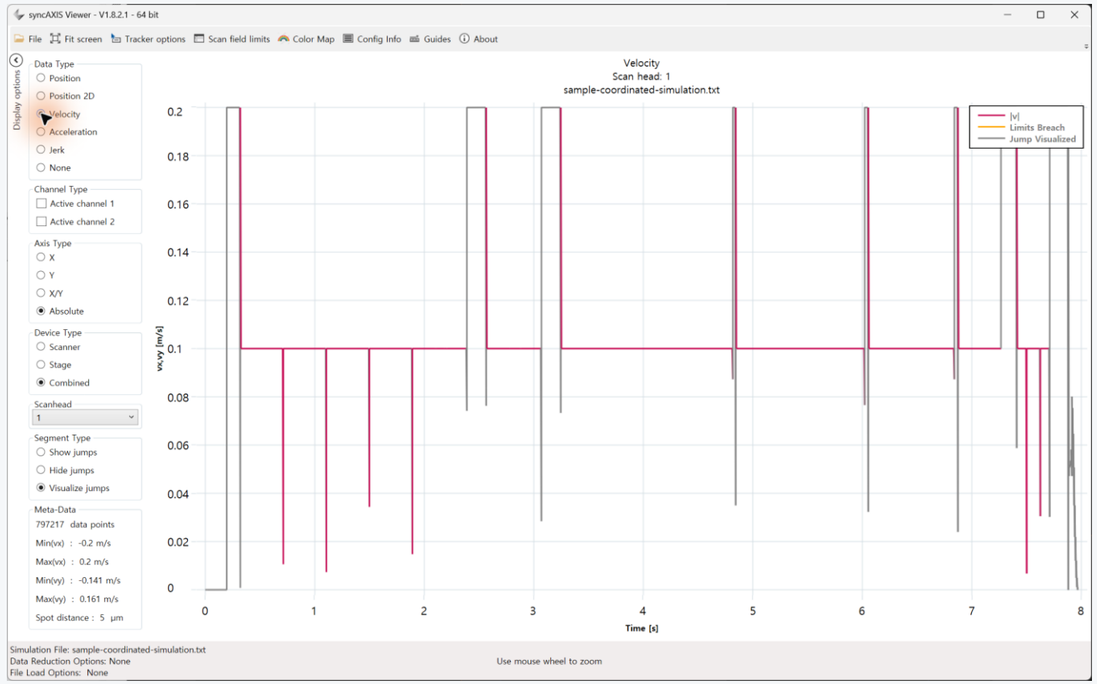
Evaluate with the required controller and encoder

# Scanner and stage share the large-area job

syncAXIS connects RTC6 and ACS motion in an XL-SCAN configuration




Archived syncAXIS simulation trajectory

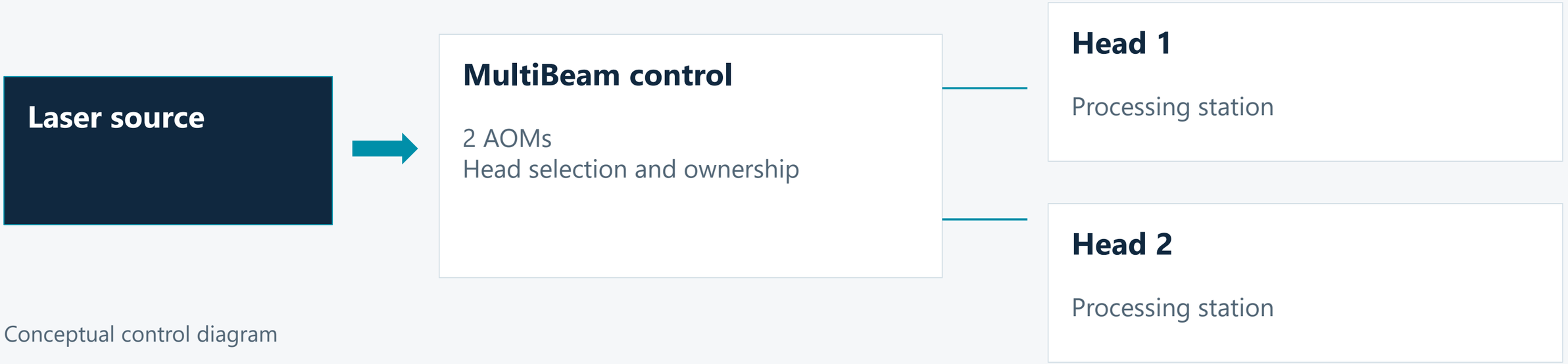



Archived velocity-profile analysis

## Coordinated motion with trajectory and dynamics analysis

# Two heads can share one laser source

MultiBeam coordinates source ownership with a two-AOM configuration

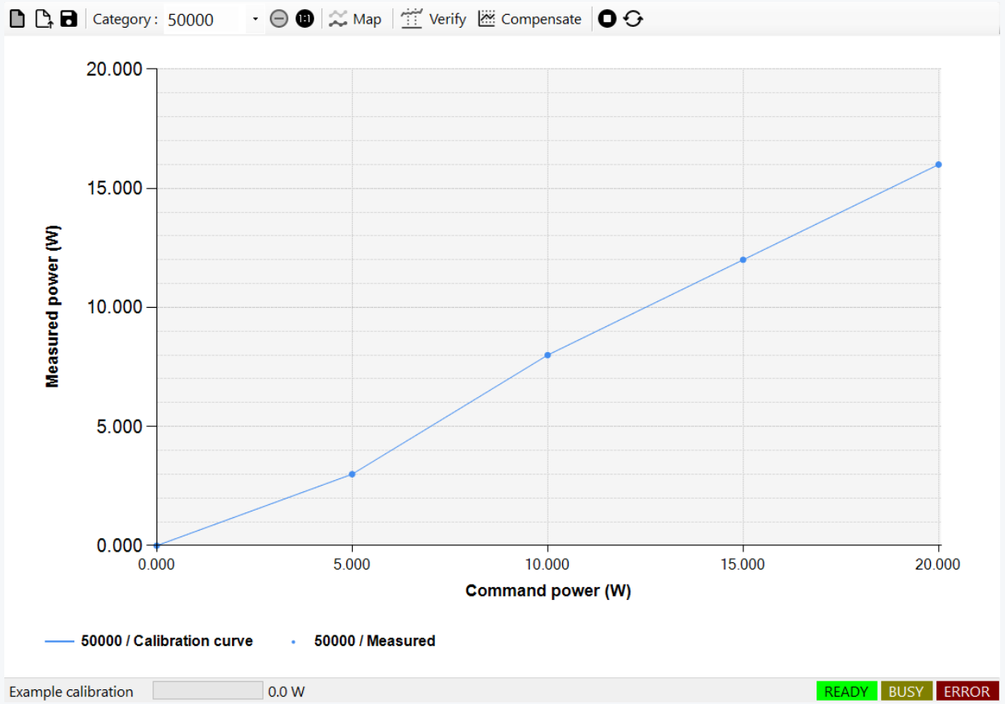



Conceptual control diagram

| Operation | What it provides |
| --- | --- |
| **Head 1 or Head 2** | Select one processing path |
| **Both** | Alternate shared-source ownership between heads |

# Power commands can use measured output data

PowerMap connects requested optical output to the required input



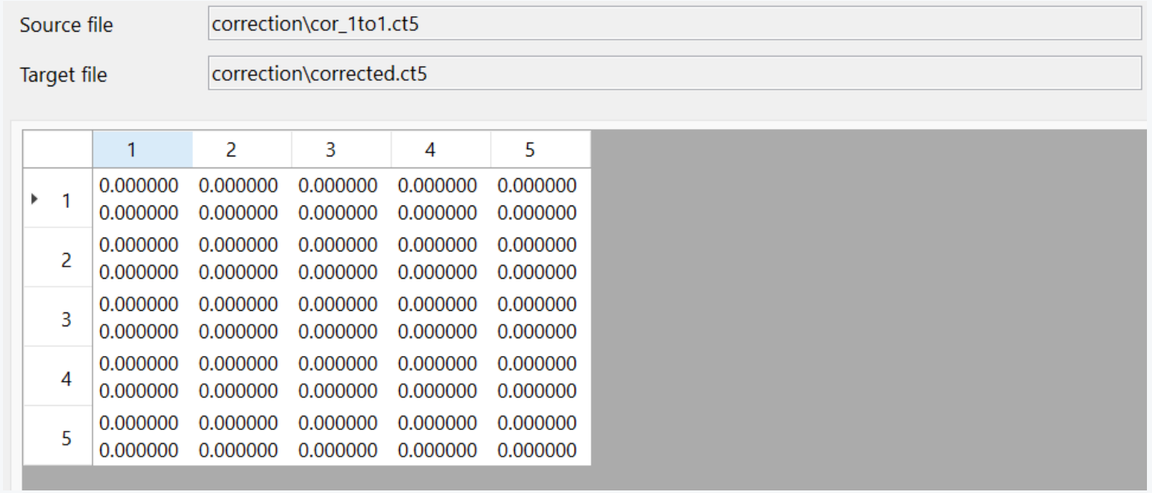

PowerMap plot with measured points

| Step | Purpose |
|---|---|
| **Mapping** | Acquire input/output measurement pairs |
| **Verify** | Check target output against the stored map |
| **Compensate** | Update mapping from measured differences |
| **Interpolation** | Choose linear or shape-preserving PCHIP |

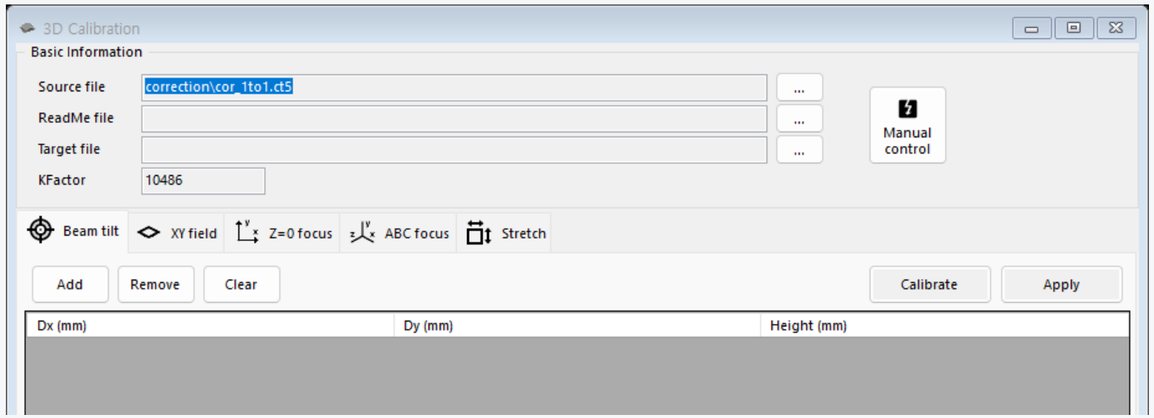
## Calibrate, verify and retain the measurement basis

# Field correction belongs beside the recipe

2D grid correction and 3D calibration support different geometries




2D measured-grid correction input



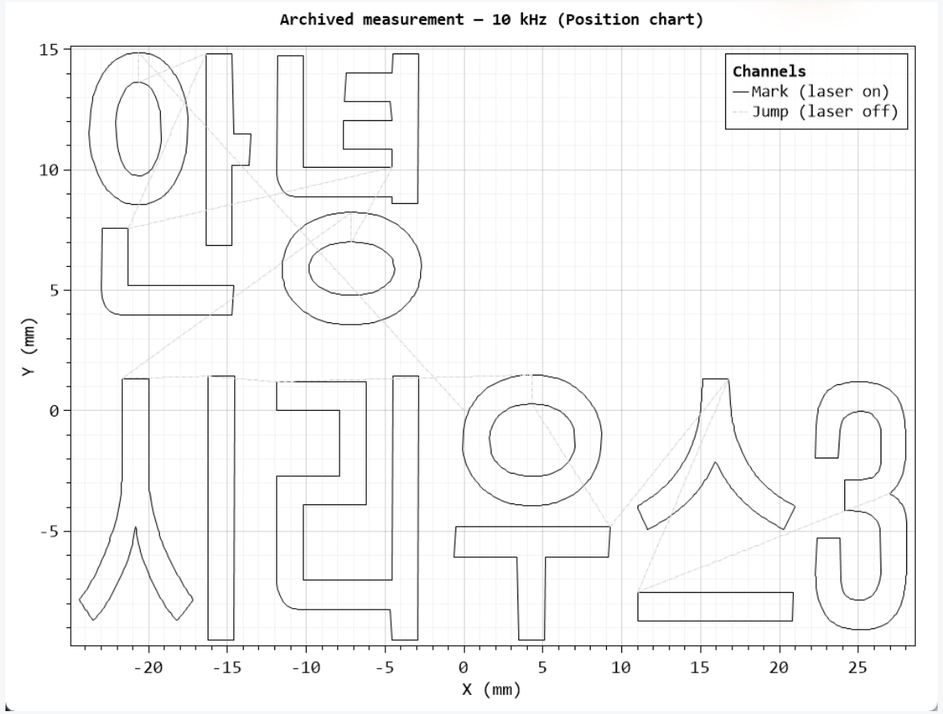

3D calibration settings

**2D correction**
**Measured field data and correction files**

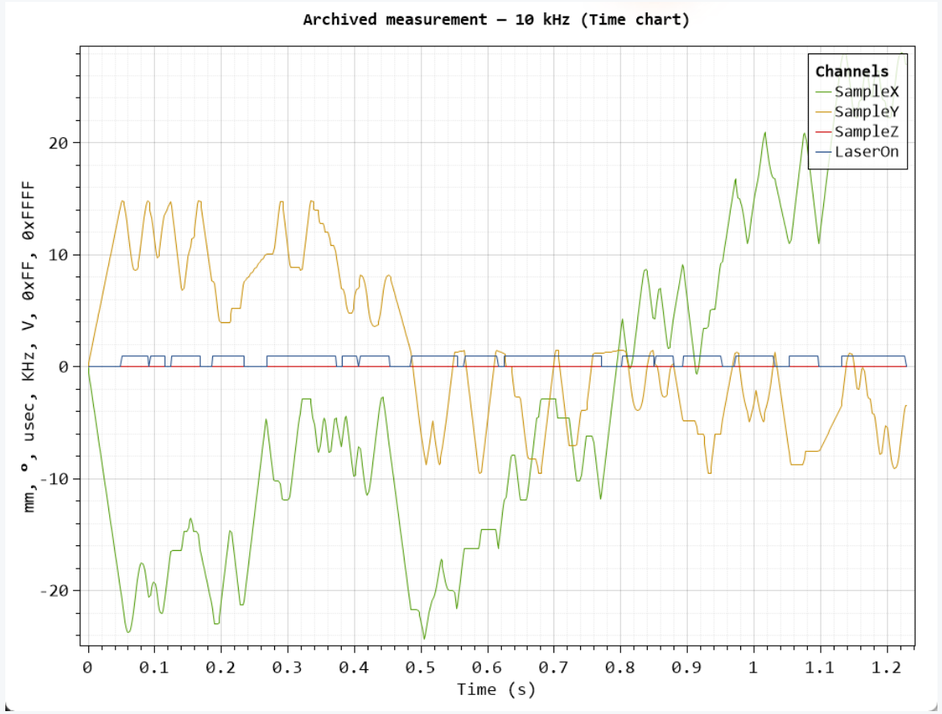
**3D correction**
**Tilt, focus, coefficients and stretch**

# The job can produce data for engineering review

Trajectory and signal plots support process investigation




Archived XY trajectory plot

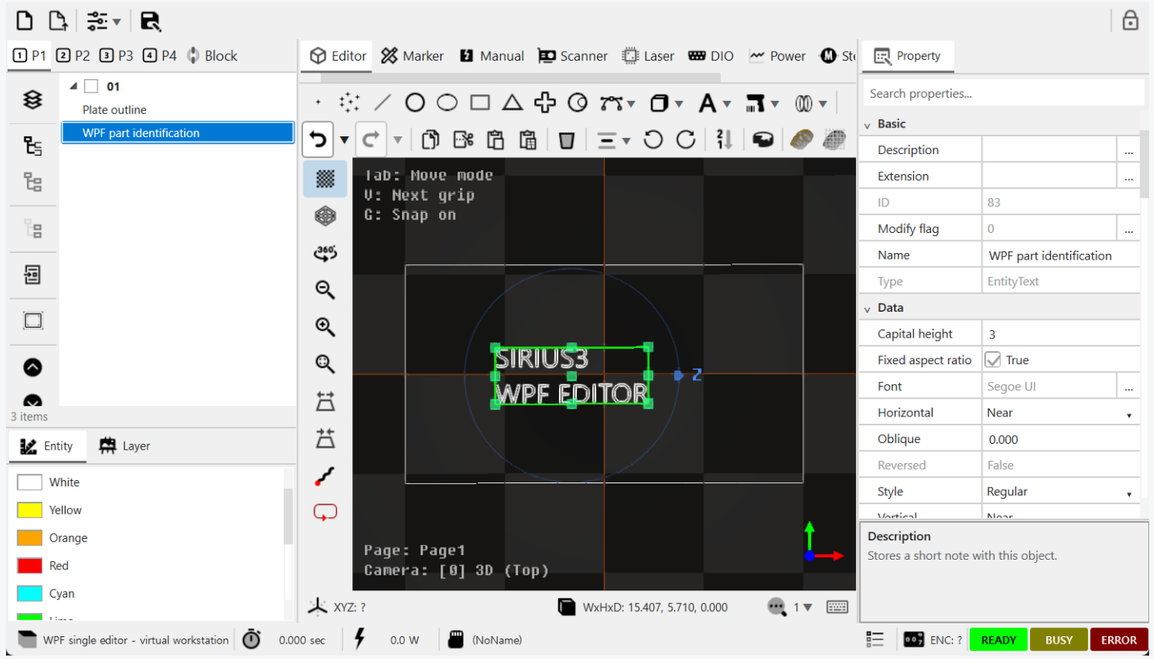



Archived signal trace

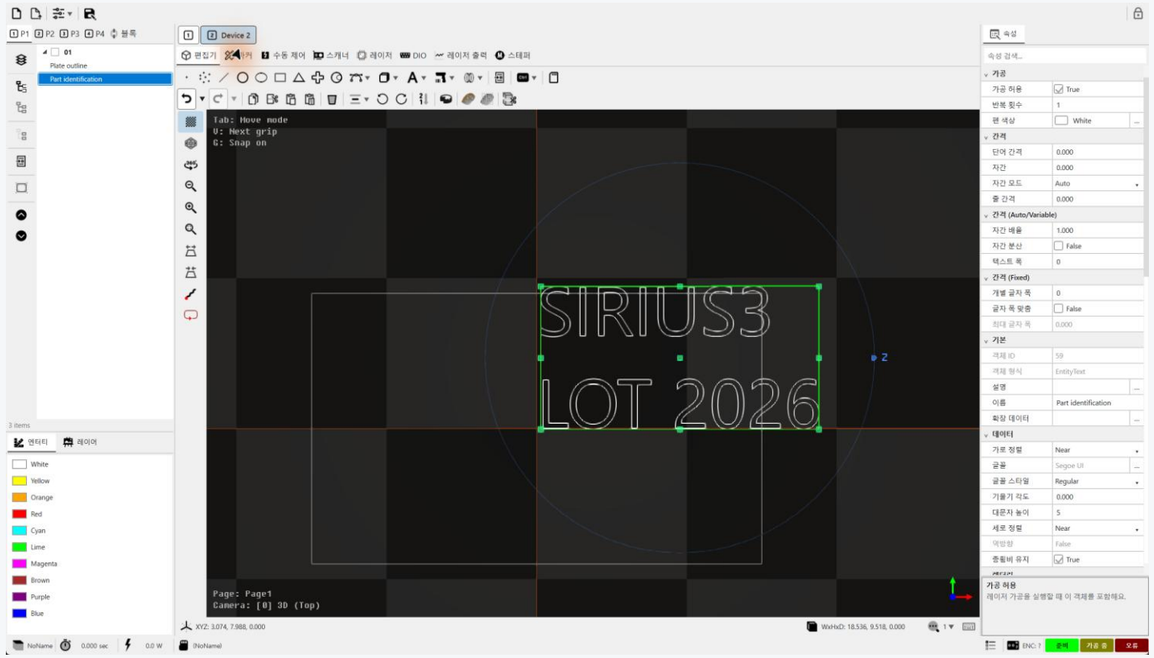
**Trace the motion, compare signals and investigate the sequence**

# Your equipment UI can use WinForms or native WPF

Both frameworks ship in the Release UI package




Native WPF SiriusEditorControl

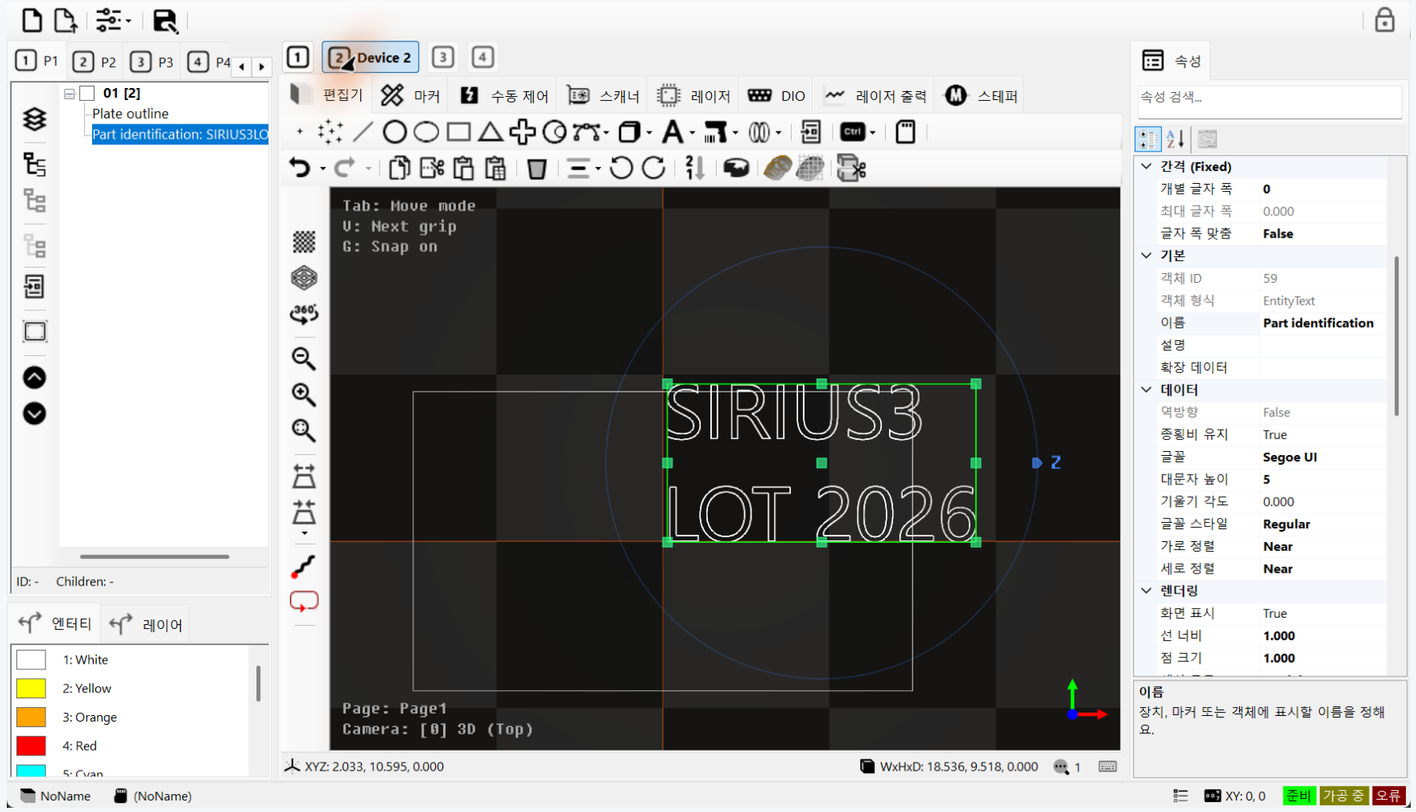



Native WPF SiriusMultiEditorControl

## Shared recipe capabilities, framework-specific UI integration

# One editor can manage multiple device sets

SiriusMultiEditorControl switches the active registered device set



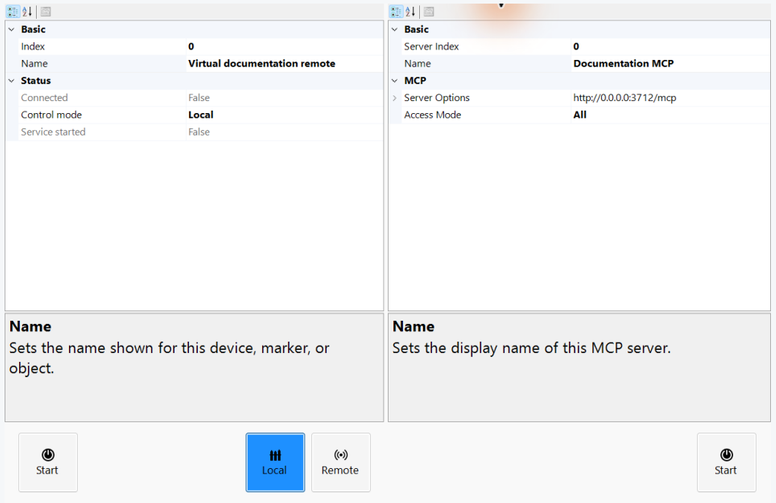

WinForms multi-device editor and active device selector

| Single editor | Multi-editor |
|---|---|
| **One assigned device set** | Indexed device-set registration |
| **Direct active-device binding** | Switch active scanner, laser and marker |
| **Shared recipe capabilities** | One document shared across registered sets |

# The recipe can join the factory data flow

The Remote option provides multiple transport choices

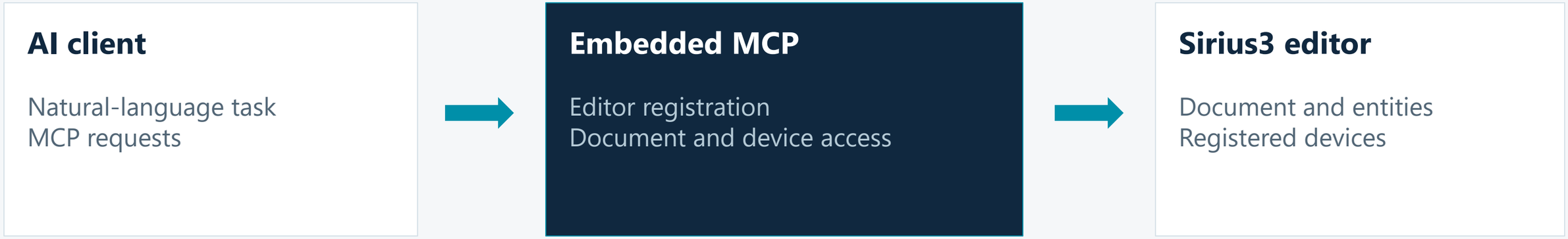
| Connection | Typical integration role |
| --- | --- |
| **TCP/IP sockets** | Equipment application and host integration |
| **Serial / RS-232** | Serial-connected host or device workflows |
| **WebSocket** | Bidirectional network application requests |
| **MQTT** | Broker-based message integration |
| **Embedded MCP** | AI-assisted document and registered-device operations |




Remote and MCP configuration panel

# AI requests can reach the working editor

An embedded MCP server binds AI clients to the caller-owned editor




| Request type | Example capability |
| --- | --- |
| **Document and entity** | Import, create, edit, clip and simulate |
| **Registered devices** | Query state and invoke supported device operations |
| **Public help** | Discover APIs and supplied documentation |

Verify MCP tool calls with the actual Release package before customer demonstrations.

# From a prompt to editable geometry

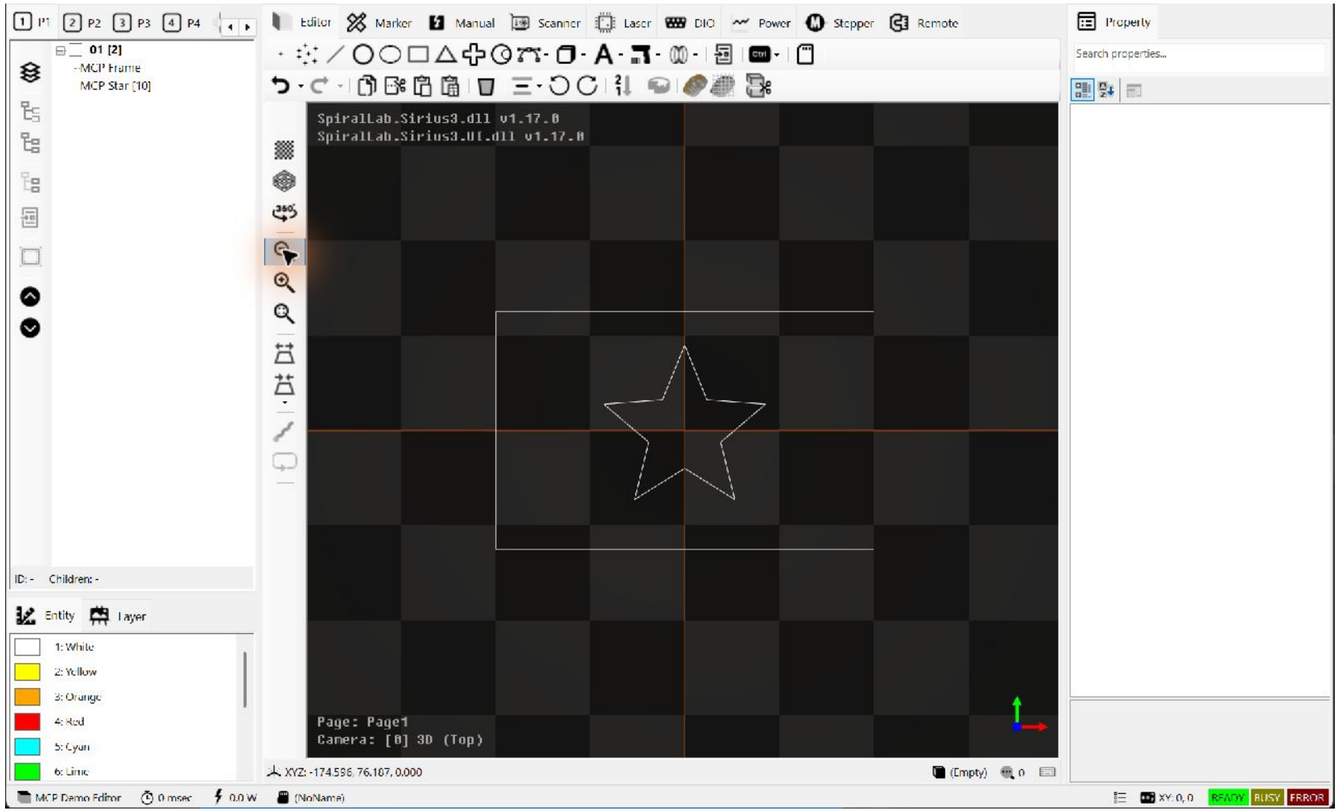
A frame and a star in the active layer

## EXAMPLE PROMPT

**Create a 40 × 25 mm rectangle at (0, 0), named MCP Frame. Add a centered five-point star named MCP Star.**

## VERIFIED RESULT

Two objects: a 40 × 25 mm rectangle and an editable 10-vertex polyline.




Actual editor after MCP tool calls

Executed MCP calls in an isolated virtual-device Debug UI build.

# A follow-up prompt edits the same object

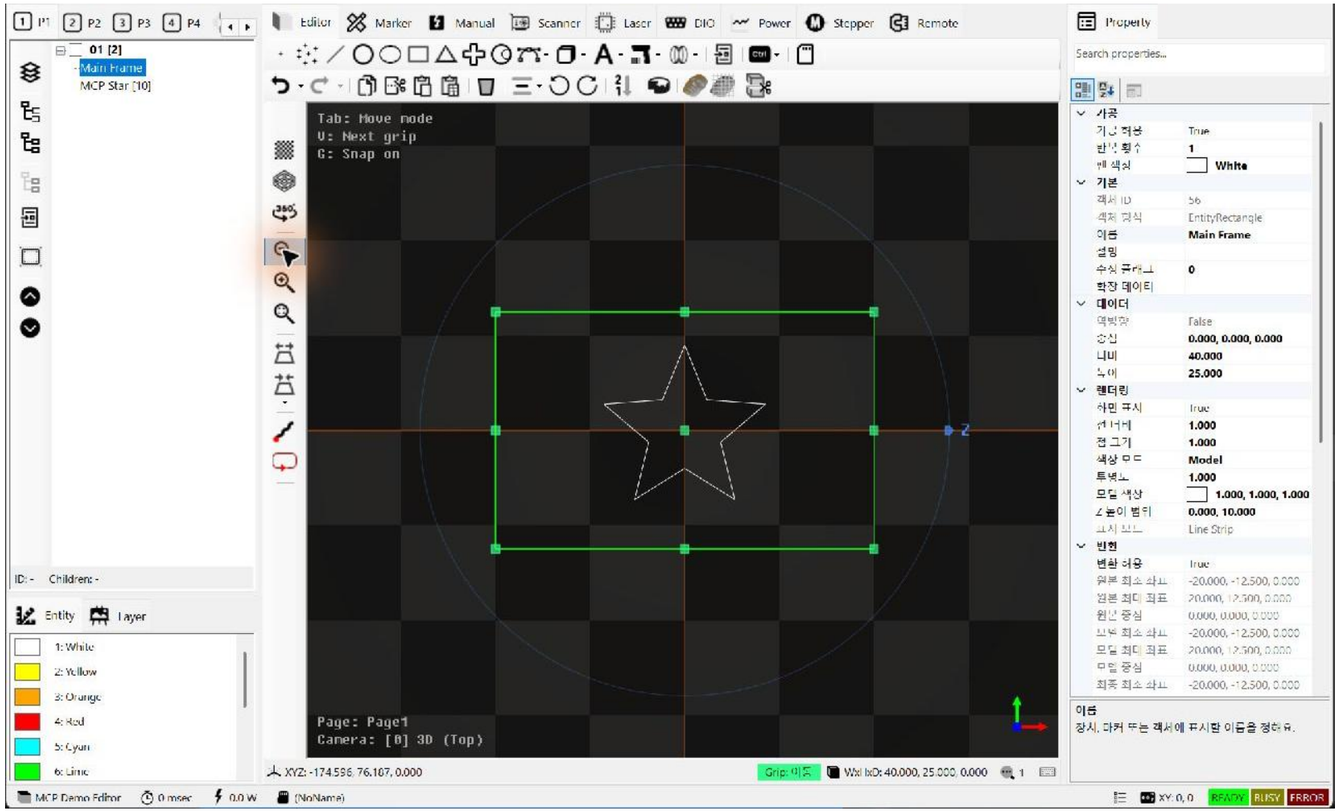
Inspect and update without leaving the document

**EXAMPLE PROMPT**

**Rename MCP Frame to Main Frame.**
**Confirm its size remains 40 × 25 mm.**

**VERIFIED RESULT**

Main Frame is selected. Its name and dimensions appear in the property panel.




Actual editor after the MCP property change

Executed MCP calls in an isolated virtual-device Debug UI build.

# Your process logic can stay your own

Public interfaces and factories connect custom equipment and application behavior

**SIRIUS3**

Document and geometry
Control and rendering

→ **Custom devices**

Scanner, laser and power meter

→ **Custom execution**

Marker and PowerMap implementations

→ **Custom experience**

Entity, editor UI, script and remote

**Reuse the common platform and specialize where the equipment differs**

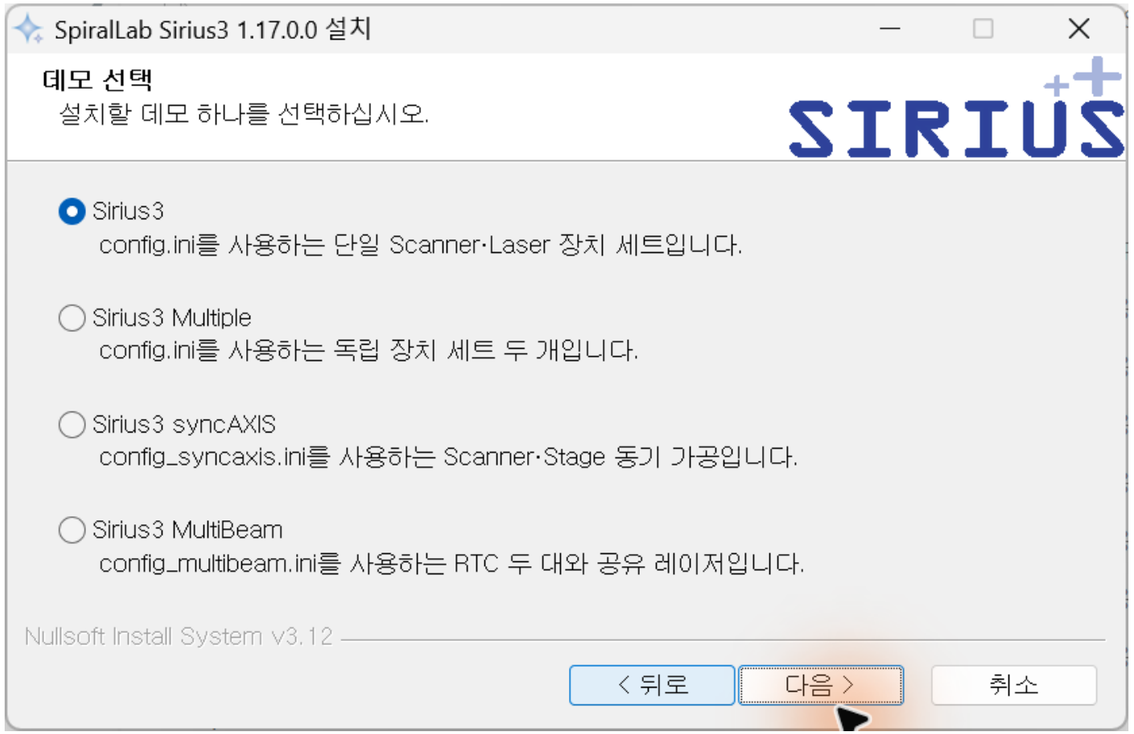
# A modern .NET integration path

NuGet packages, public demos and multilingual documentation

| Area | Supported foundation |
| --- | --- |
| **Runtime targets** | .NET Framework 4.8.1, .NET 8 / 9 / 10 for Windows |
| **UI frameworks** | WinForms and native WPF in Release |
| **Package structure** | Core, UI and Dependencies NuGet packages |
| **Visualization** | OpenGL 3.3+ with a compatible GPU/driver |
| **Public demos** | Visual Studio 2022+, net481 / net8 / net9 Windows |
| **Localization** | English, Korean, Chinese, Japanese and German resources |

# Install the demo that fits your system

Download the Windows installer from GitHub Releases and select one of four demos.




Actual installer selection screen (Korean)

| Demo | Configuration |
|---|---|
| **Sirius3** | One scanner/laser set<br>config.ini |
| **Sirius3 Multiple** | Two independent device sets<br>config.ini |
| **Sirius3 syncAXIS** | Scanner + stage synchronization<br>config_syncaxis.ini |
| **Sirius3 MultiBeam** | Two controllers, one shared laser<br>config_multibeam.ini |

**GitHub Releases: github.com/labspiral/sirius3/releases**

# The license follows the equipment configuration

RTC instances with the extension options the process needs

| Scope | What it covers | Typical reason to select |
|---|---|---|
| **Base RTC instance** | Common scanner/laser and recipe foundation | Standard equipment integration |
| **MoF option** | Encoder-based moving-workpiece compensation | Linear or rotary moving workpieces |
| **syncAXIS option** | Scanner/stage coordination | XL-SCAN large-area configurations |
| **MultiBeam option** | One source, two AOMs and two heads | Shared-source dual-head systems |
| **Remote option** | TCP, serial, WebSocket, MQTT and MCP | External host and AI integration |

Commercial use requires the appropriate license. Vendor hardware, runtime and licenses are configuration dependent.

# Your process.
# A focused integration plan.

**01**

### Bring a real sample

CAD, codes or 3D data

**02**

### Define the configuration

RTC, laser, head, motion and options

**03**

### Evaluate the workflow

Edit, simulate and validate on the system

**hcchoi@spirallab.co.kr**

spirallab.co.kr

Public demos: github.com/labspiral/sirius3